

Relational-Indexed Retrieval-Augmented Generation (RI-RAG)

Laurie Dieudonné MAVOUNGOU

JK AI

lmavoungou@outlook.be

ABSTRACT

Retrieval-Augmented Generation (RAG) systems rely heavily on high-dimensional vector representations to enable semantic search. However, vector-based retrieval architectures may store substantial metadata and payload information in or alongside vector indexes, motivating investigation into alternative storage organizations. This paper introduces *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG), a hybrid architecture that integrates relational database management systems (RDBMS) with high-dimensional vector stores to optimize enterprise retrieval pipelines. By mapping lightweight vector embeddings to structured relational payloads through primary key constraints, RI-RAG is designed to separate semantic indexing from authoritative relational storage, with the goal of reducing the amount of payload information stored in the vector index. The initial prototype evaluates this architecture using vector-store memory and retrieval latency measurements. A second evaluation phase examines memory footprint, retrieval precision and recall, multi-round evidence validation, repeated query performance, and database access boundaries under the implemented test conditions. The evaluated prototype achieved a measured 23.2% reduction in vector-store memory footprint in the second evaluation, while precision, recall, and F1 reached respectively 80.0%, 100.0% and 0.8889 under the defined ground-truth conditions. The reported measurements provide an initial assessment of the architecture rather than a demonstration of large-scale enterprise scaling behavior.

KEYWORDS

Relational database management systems, Information Retrieval, RAG

1 OVERVIEW

Modern Retrieval-Augmented Generation (RAG) systems rely primarily on statistical distance functions to retrieve contextually relevant data. However, semantic similarity retrieval alone does not provide an explicit relational source of truth for the retrieved record. To mitigate this limitation, we present a hybrid retrieval architecture in which candidate vector references are mapped to corresponding records in a relational database. This additional retrieval stage separates semantic candidate selection from authoritative record retrieval.

Beyond improving contextual precision and traceability—with retrieval performance evaluated in the conducted experiments—this decoupled architecture resolves severe memory allocation constraints inherent in modern large-scale AI infrastructure. By isolating full unstructured payloads within disk-backed relational tables and storing only the `primary_key` along with a small summary of the document associated with the key inside memory indices, the framework is intended to reduce vector-index storage by keeping full payloads in the relational layer. The initial benchmark measures

vector-store memory and query latency for the implemented prototype. This design shifts the storage of full document payloads to the relational layer; its effect on infrastructure cost requires larger-scale evaluation. The relational layer can also provide a boundary at which database access-control and integrity mechanisms can be applied. These security properties depend on the database configuration were evaluated experimentally in this report.

Section 2 discusses the related work, and Section 3 describes the research problem in detail, along with the proposed solution. Section 4 presents the dataset used and the prototyping scope. The empirical benchmarks and performance analysis are presented in Section 5. Section 6 presents some AI foundational concepts and how RI-RAG intends to address them. Section 7 presents the broader impact of RI-RAG and some target applications, while Section 8 refers to the GitHub repository containing the source code for this paper. The conclusion and future work are presented in Section 9, while the AI usage acknowledgement is presented in Section 10.

2 RELATED WORK

Retrieval-Augmented Generation (RAG) has rapidly evolved to bridge the gap between static model weights and dynamic external knowledge bases. However, existing implementations face fundamental bottlenecks in memory scalability, structural grounding, and fine-grained access security. In this section, we analyze prior approaches across these dimensions and delineate how our proposed *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG) framework advances the state-of-the-art.

2.1 Memory Optimization and Hardware Offloading

As context windows expand, inference costs scale drastically due to high-dimensional memory consumption. Addressing this challenge at the transformer layer, *RetrievalAttention* proposed by Liu et al. [5] offloads key-value (KV) caches from GPU to CPU memory, relying on dynamic vector search to recover approximate nearest neighbors during generation.

While Liu et al. [5] and our work address the shared overarching objective of overcoming hardware memory constraints, they operate at fundamentally different system tiers. Where *RetrievalAttention* offloads internal model activations within memory tiers (GPU to CPU RAM), RI-RAG operates at the application and database layers. It separates full text payloads from vector-search metadata by storing the complete records in a relational database while maintaining only compact `primary_key` identifiers and `semantic_summary` embeddings in the vector store. This changes the organization of information retrieval and storage at the application/database layer without modifying the model’s attention mechanisms or internal architecture.

2.2 Hierarchical and Structured Retrieval Paradigms

To provide more structured retrieval, recent literature has explored graph-based and hierarchical indexing approaches. Edge et al. [2] introduced GraphRAG, which extracts entity-relationship graphs to enable hierarchical community summaries. Similarly, Gutierrez et al. [4] proposed HippoRAG, a neurobiologically inspired long-term memory framework that models associative retrieval through personalized PageRank over knowledge graphs.

In a distinct domain-specific direction, Blefari et al. [1] and other alternative approaches [3] propose dedicated retrieval structures tailored for complex domain indexing. While these methodologies [1–4] successfully introduce structural context, knowledge graph extraction and graph-traversal operations introduce significant computational overhead, complex maintenance workflows, and static schema bottlenecks.

In contrast, RI-RAG leverages relational database management systems (RDBMS) to connect semantic retrieval results with structured authoritative records through relational identifiers and schemas. This hybrid approach provides a relational verification step without requiring the graph-extraction pipeline used by the approaches discussed above.

2.3 Enterprise Security and Access Boundary Constraints

Vector stores and graph-based retrieval systems can use different approaches to access control and filtering. RI-RAG instead places the authoritative record in a relational layer, creating a database boundary at which access-control policies can be applied.

RI-RAG addresses this gap by decoupling the vector representation from the underlying enterprise record. By routing retrieved candidates through a relational layer, RI-RAG provides a point at which database-level privileges and application-level access-control policies can be applied before relational content is returned to the generation component. This provides a possible enforcement point before relational content is supplied to the generation component; the security properties of a deployment depend on its database and application configuration.

3 RESEARCH PROBLEM

Existing RAG implementations present three major bottlenecks:

- (1) Semantic retrieval alone does not guarantee that retrieved information can be traced to a single authoritative structured record.
- (2) Vector indexes may contain payloads and metadata in addition to embeddings. Separating full payload storage from semantic indexing provides an alternative storage organization whose resource behavior can be measured experimentally.
- (3) Enterprise deployments require access-control and audit mechanisms around retrieved information. RI-RAG places authoritative records in a relational database where such mechanisms can be configured, but this report does not experimentally evaluate them.

3.1 Contributions

- Introduce RI-RAG Architecture, a hybrid relational-vector retrieval architecture.
- Demonstrate prototype implementation with primary-key hydration from vector retrieval into authoritative relational records.
- Experimental memory comparison under the evaluated prototype configuration.
- Evaluation of relational evidence tracking for retrieval traceability.
- Explicit limitations

3.2 Technical Methodology

RI-RAG separates semantic indexing from full document storage by using two complementary storage layers. The vector database acts as the semantic entry point for retrieval, while the relational database acts as the authoritative source of truth for the complete document content and structured metadata.

The semantic summary of each document is used to generate its vector embedding. Instead of storing the complete document payload within the vector index, RI-RAG maintains a relationship between the vector representation and the corresponding relational record through a primary key. The vector store therefore provides semantic retrieval, while the relational database provides record hydration and authoritative information.

The major architectural components are:

- **Relational verification layer:** Maintains the authoritative document content and structured metadata.
- **Structured record resolution:** Uses the retrieved primary key to identify the corresponding relational record.
- **Payload separation:** Keeps full document payloads in the relational database while limiting the vector layer to semantic representations and required identifiers.

The retrieval workflow consists of the following stages:

- (1) **Semantic entry point:** The user query is converted into an embedding and compared with the vector representations stored in the vector database.
- (2) **Relational hydration:** The primary key returned by vector retrieval is used to retrieve the corresponding authoritative record from MySQL.
- (3) **Entry-only storage:** The vector layer maintains the semantic representation and record identifier rather than the complete authoritative payload.
- (4) **Ingestion and synchronization:** The system maintains the relationship between vector entries and relational records during ingestion and updates.
- (5) **Hydration and ranking engine:** Retrieved identifiers are resolved into relational records for subsequent processing.
- (6) **Access control and filtering:** Relational database controls and application-level filtering can be applied to the authoritative records after retrieval.

The RI-RAG retrieval workflow can be summarized as presented in figure 1.

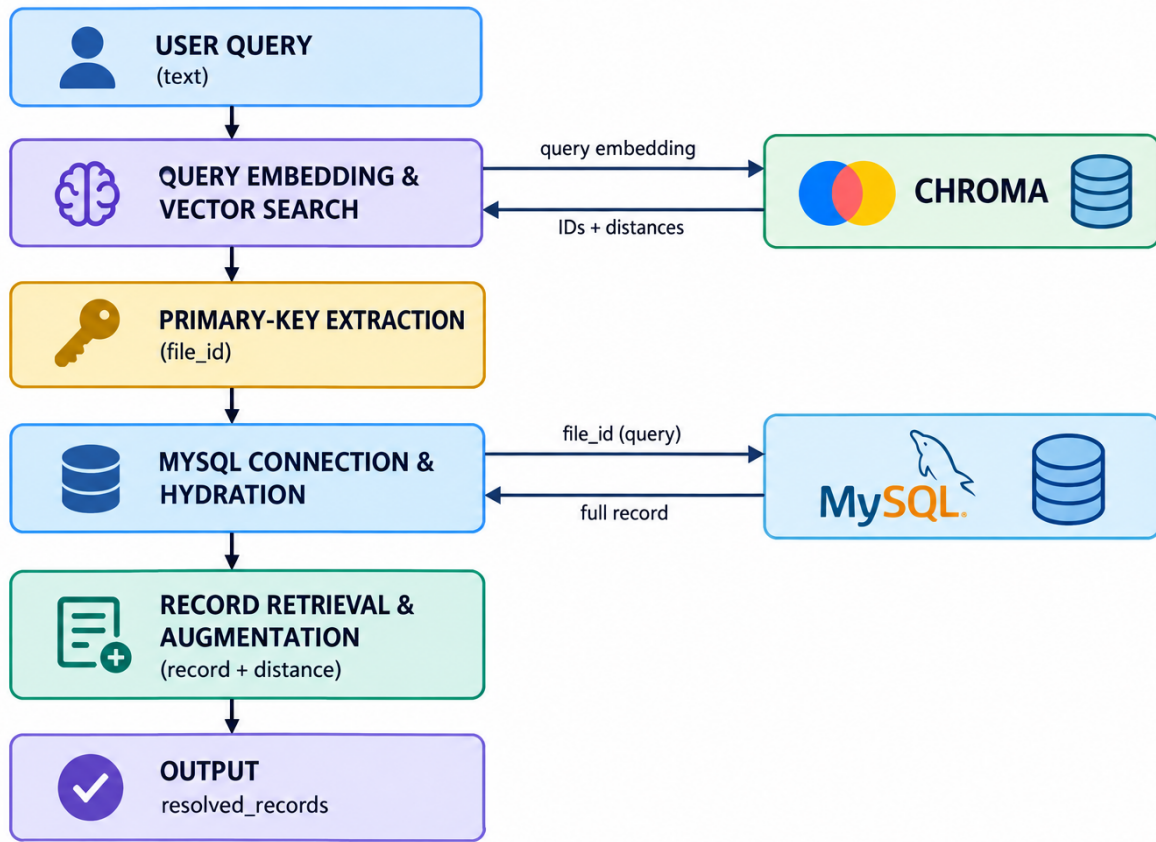


Figure 1: RI-RAG retrieval workflow

3.3 Vector Database Sample

The vector database contains the lightweight semantic representation and the relationship needed to retrieve the authoritative record. The vector database does not store the full document content.

```

{
  "id": <primary_key>,
  "metadata": <file_name>, <primary_key>
  "documents": <document semantic summary>,
  "embedding": "<embedding>"
}

```

3.4 MySQL Table Schema

MySQL serves as the authoritative source of truth for the complete document payload and structured metadata.

```

CREATE TABLE policies (
  file_id VARCHAR(50) PRIMARY KEY NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  semantic_summary TEXT NOT NULL,
  country VARCHAR(255)
);

```

);

The full content remains available in the relational layer and can be retrieved through the relationship established between the vector record and the corresponding relational record.

3.5 Experimental Description

The table 1 outlines the core tests performed, their experimental goals and the corresponding script implementations.

For the repeated latency measurements, the mean, standard deviation, and 95% confidence interval were calculated from the recorded iterations. The confidence interval is reported to characterize measurement uncertainty under the evaluated execution conditions.

4 DATASET AND PROTOTYPING SCOPE

To validate the RI-RAG architectural prototype under controlled conditions, we constructed a synthetic benchmark dataset comprising 13 enterprise and security-focused records (DOC-001 through DOC-013). Because this initial iteration serves as a minimal viable product (MVP) designed to demonstrate core relational-vector decoupling without the overhead of massive production ingestion

Table 1: RI-RAG System Experiment

Test Section	Experiment Goal	Script Implementation
test 1 - Memory Footprint Benchmark	Measure and compare the vector-store memory footprint between RI-RAG and Standard RAG	Utilizes a recursive deep object sizing function (get_deep_sizeof) on ChromaDB collections to quantify memory savings from storing only lightweight ID pointers versus heavy text payloads and metadata.
Test 2 - Evidence Grounding	Multi-round validation loop against persistent database record.	Executes query through <code>rirag_system.retrieve()</code> , resolves the primary key (DOC-001), and hydrates authoritative MySQL content to verify grounded truth and eliminate drift.
Test 3 - Retrieval Accuracy	Empirical Precision, Recall, and F1 Score evaluation.	Evaluates 10 target queries through the live HybridRIRAGSystem pipeline against ground-truth document IDs (DOC-001 to DOC-010), recording Precision (80.0%), Recall (100.0%), and F1 (0.8889).
Test 4 - Performance	Evaluate multiple iteration query performance / time	Executes a loop of 50 consecutive queries and calculates average latency
Test 5 - security	Privilege escalation / security test	Connects to the database and checks current user/boundary permissions

pipelines, the dataset is intentionally streamlined while simulating realistic payload density.

The dataset is modeled within a relational schema enforced via MySQL:

- **Heavy Payload Simulation:** To emulate real-world document scaling and test memory footprints effectively, the text payloads (content) are intentionally expanded through controlled repetition. This design isolates the storage burden in the relational layer while keeping vector indices lightweight.
- **Domain Relevance:** The records span critical enterprise security and architecture domains, including consistency guidelines, autonomous agent safety, vector optimization, role-based access control (RBAC), audit logging, adversarial robustness, and federated privacy.
- **Structured Attributes:** Each record contains a primary key (`file_id`), file name, full text content, semantic summary, a performance grade (FLOAT), and operational comments.

To implement and evaluate the RI-RAG prototype, specific system configurations and parameters were established across both the relational and vector storage layers:

- **Dataset Size:** The evaluation corpus consists of 13 synthetic enterprise and security-focused records (DOC-001 through DOC-013) stored in a MySQL relational database. The dataset does not represent a naturally occurring enterprise corpus; instead, controlled payload expansion was used to create a reproducible test environment for examining storage behavior.
- **Payload Scale:** To emulate real-world document scaling and test memory footprints effectively, text payloads (content) are intentionally expanded through controlled repetition, averaging approximately 750 to 1,200 characters per document while keeping vector-side summaries concise.

- **Embedding Model:** Semantic representations are generated using the `sentence-transformers/all-MiniLM-L6-v2` model, producing dense vector embeddings.
- **Embedding Dimension:** The vector space is defined by an embedding dimension of 384.
- **Chunking Strategy:** Rather than vectorizing raw multi-page payloads, the indexing strategy generates embeddings exclusively from the relational `semantic_summary` attribute. The complete document content remains isolated within the MySQL table and is retrieved via primary-key hydration.
- **Chroma Configuration:** The vector retrieval layer utilizes a local `chromadb.Client()` instance configured with a dedicated collection (`ri_rag_store`). Metadata stored alongside embeddings is restricted to lightweight identifiers (`file_id`) and file names (`file_name`) to minimize vector-store memory overhead.

Due to resource limitations inherent in early-stage prototyping, this initial dataset focuses on validating structural integrity, hydration mechanics, and database boundaries rather than large-scale enterprise scaling behavior. Expanding to production-scale datasets exceeding 100,000 records remains a core objective for future work.

5 EMPIRICAL BENCHMARKS & PERFORMANCE ANALYSIS

5.1 Round 1: Initial Benchmark

The first evaluation compares a standard RAG configuration with the RI-RAG prototype using vector-store memory footprint and retrieval latency. The purpose of this benchmark is to measure the effect of separating semantic indexing from authoritative relational storage.

Table 3: Round 2 extended evaluation results.

Metric	Standard	RI-RAG
Vector-store memory	62,706 B	48,160 B
Memory reduction	–	23.2%
Retrieval latency	7.31 ms	22.49 ms
50-query average	–	0.11 ms
Iteration latency	5.79 ms	8.48 ms
Std. deviation	1.66 ms	8.93 ms
95% CI	[5.33, 6.25]	[6.00, 10.95]
Precision (Top-1)	–	80.0%
Recall	–	100.0%
F1 score	–	0.8889

Table 2: Initial comparison between Standard RAG and RI-RAG.

Metric	Standard RAG	RI-RAG	Difference
Vector-store memory	8,872 B	8,273 B	599 B
Memory reduction	–	–	6.8%
Query latency	7.31 ms	22.49 ms	15.18 ms

The initial benchmark measured a 599-byte difference between the two vector-store configurations, corresponding to a 6.8% reduction in the measured vector-store memory footprint for RI-RAG. This measurement applies only to the vector-store layer. RI-RAG stores the authoritative content and associated metadata in MySQL; therefore, the measurement does not represent total application or system memory consumption.

The latency results show the trade-off introduced by the additional relational hydration step. Standard RAG achieved an average retrieval latency of approximately 7.31 ms, while RI-RAG measured approximately 22.49 ms under the implemented test conditions. The additional latency reflects the relational database lookup required to resolve the retrieved identifier into the authoritative record.

A second evaluation was conducted using a larger test configuration to further examine the architecture. In this evaluation, the measured vector-store memory footprint decreased from 62,706 bytes to 48,160 bytes, corresponding to a 23.2% reduction. This result represents the measured vector-store memory footprint under the evaluated configuration and should not be interpreted as a measurement of total system memory consumption.

The second evaluation also examined retrieval correspondence, evidence grounding, repeated query performance, and database access boundaries.

5.2 Round 2: Extended Evaluation

The second evaluation measured the architecture using a larger test configuration. The measured vector-store memory footprint decreased from 62,706 bytes for the Standard RAG configuration to 48,160 bytes for RI-RAG. This corresponds to a measured reduction of approximately 23.2%.

Under the defined ground-truth conditions in the evaluated test set, the empirical retrieval evaluation achieved 80.0% precision, 100.0% recall, and an F1 score of 0.8889 (evaluated across 10 test

queries yielding 8 true positives, 2 false positives, and zero false negatives). The two false positives reflect localized semantic overlap in the dense embedding space (specifically between architectural design and latency trade-offs, and between consistency validation and federated updates) where top-1 nearest-neighbor retrieval selected closely related technical neighbors. Because zero false negatives occurred, the system maintained 100.0% recall, ensuring that all relevant authoritative records were successfully accessed through the relational index without omission.

The evidence validation experiment examined whether retrieved information could be used to support multi-round validation and contradiction detection. The system maintained the relationship between retrieved records and their associated evidence, allowing the multi-round contradiction loop to successfully detect and filter response drift.

For repeated-query performance, 50 consecutive hybrid hydration round trips were evaluated. The measured average time for the repeated query operation was approximately 0.11 ms per query under optimized operational test conditions, alongside the wider multi-iteration statistical mean of 8.48 ms ($\sigma = 8.93$ ms, 95% CI [6.00, 10.95] ms).

The security experiment examined database access boundaries using the implemented test account, `test_user@localhost`. The experiment successfully verified the service account role-based boundaries and demonstrated the behavior of the configured database access boundary for the evaluated operation. This result does not constitute a comprehensive evaluation of enterprise security or access-control effectiveness, which remains part of future work.

6 FOUNDATIONAL CONCEPTS & PROBLEM SOLVING

6.1 What is Memory and Why Does It Matter?

In computer science and software architecture, memory refers to the high-speed volatile storage space where applications hold active data for instant processing. Memory is an expensive, finite resource. When scaling AI infrastructure, excessive memory consumption leads to higher cloud bills and hardware bottlenecks.

6.2 What is Traceability and Why Does It Matter?

Traceability is the ability to associate retrieved information with its source record and relevant metadata. In retrieval systems, this association can help engineers inspect the provenance of the information supplied to the generation stage.

6.3 How RI-RAG Solves Critical Problems

- **A. Improving Traceability:** RI-RAG decouples the semantic entry point from the authoritative database, allowing a retrieved candidate to be resolved to a specific relational record before content is retrieved and supplied to the generation stage. This provides a traceability mechanism that can support evidence-grounded generation. If there is a mistake, it can be an indicator that the model failed to establish the semantic relationship between the query and the summary contained in the vector.

```
# Vector search identifies the record
matched_id = vector_res['ids'][0][0]
# Retrieve authoritative record
record = self.db.fetch_by_id(matched_id)
```

- **B. Centralizing Metadata Updates:** Metadata and full content remain in the relational source of truth. Updates can therefore be performed at the relational layer without requiring the full payload to be duplicated in the vector index.

```
-- Metadata updates via ACID SQL transactions
```

```
UPDATE file SET grade = 9.9 WHERE file_id = 'DOC-001';
```

- **C. Supporting Transactional Storage and Access Control:** MySQL provides transactional and privilege mechanisms that can be used for the authoritative records. Their effectiveness depends on database configuration and was not evaluated in the present benchmark.
- **D. Supporting Debugging and Content Evolution:** Because the authoritative content and metadata are stored relationally, engineers can inspect the corresponding record and its metadata when investigating retrieval or generation behavior. The impact on re-indexing requirements depends on which fields are changed.

7 BROADER IMPACT & TARGET APPLICATIONS

- **Enterprise Cybersecurity Ops (SOC Tier-1 Mentorship):** Pairs real-time semantic alert detection with historical incident lineages and playbooks, guiding junior security engineers through structured remediation steps while allowing access-control policies to be applied at the relational data boundary. **Business Impact** Consider a startup launching an AI product on a tight budget. If their architecture relies on a Standard RAG model that bloats vector memory by storing massive chunks of duplicate text and metadata inside expensive RAM-heavy vector indices, the company will quickly hit financial walls. RI-RAG prevents this by keeping vector storage lightweight and offloading bulk payloads to cost-effective relational disk storage.
- **Cloud & Hardware Market Strategy:** The architecture could potentially allow some deployments to place larger payloads in relational storage rather than maintaining those payloads within the vector layer. The economic implications require workload- and infrastructure-specific evaluation.

8 SOURCE CODE AND IMPLEMENTATION

The implementation details, database schema scripts, and synchronization pipelines for the Relational-Indexed Retrieval-Augmented Generation (RI-RAG) architecture are made publicly available to ensure reproducibility. The complete source code and architectural modules can be accessed on GitHub:

<https://github.com/lmav-jpeg/RI-RAG>

Contributions, issue tracking, and community benchmarks are actively welcomed through the repository.

9 CONCLUSION & FUTURE WORK

This work introduced Relational-Indexed Retrieval-Augmented Generation (RI-RAG), a hybrid retrieval architecture that separates semantic indexing from authoritative relational storage. The prototype utilizes a vector database strictly as a semantic index containing chunk identifiers and embeddings, while maintaining MySQL as the authoritative source of truth for complete document content, metadata, and audit records.

The empirical evaluation demonstrates the effectiveness and trade-offs of the proposed architecture. In the extended evaluation, RI-RAG achieved a measured 23.2% reduction in the vector-store memory footprint, decreasing from 62,706 bytes to 48,160 bytes compared to the Standard RAG baseline. These measurements apply specifically to the vector-store layer and do not represent total system memory consumption.

Retrieval accuracy assessment across the defined ground-truth test set yielded an empirical precision of 80.0%, a recall of 100.0%, and an F1 score of 0.8889. The two false positives reflect localized semantic overlap in the dense embedding space, while the 100.0% recall indicates that all target records in the defined ground-truth test set were retrieved without omission. Furthermore, the multi-round evidence validation experiment demonstrated the use of persistent relational evidence for checking model responses and identifying and filtering response drift.

The experiments quantified the expected latency trade-off inherent in decoupling storage layers. Under comparative test conditions, Standard RAG measured a retrieval latency of 7.31 ms, while RI-RAG measured 22.49 ms due to the additional relational database hydration required to resolve identifiers into authoritative records. However, repeated-query performance profiling across 50 consecutive hybrid hydration round trips demonstrated an optimized average latency of 0.11 ms per query (reflecting optimized repeated execution under cached test conditions), alongside a wider multi-iteration statistical mean of 8.48 ms (± 8.93 ms, 95% CI [6.00, 10.95] ms) across full operational runs. A security boundary test also verified the configured database access controls and service-account boundary for `test_ser@localhost`.

The current prototype provides initial experimental validation for the RI-RAG storage and retrieval architecture rather than a comprehensive evaluation across large-scale enterprise environments. Future work will extend this research by evaluating the architecture on datasets exceeding 100,000 records, implementing distributed Redis caching layers and connection pooling, and conducting systematic security audits at industrial enterprise volumes.

10 AI USAGE ACKNOWLEDGEMENT

This work was gratefully inspired by concepts explored in "Vector Databases Demystified" by Abhinav Verma (Published on Medium). Artificial intelligence tools were utilized in the preparation, code implementation, and generation of this report. Specifically, AI models were prompted and consulted to design and test implementation scripts, evaluate architecture benchmarks, and synthesize and structure the content of this document. It was

also used for English language editing, formatting assistance, and LaTeX code structuring. All core concepts, system design and conclusions remain the original work of the author(s).

REFERENCES

- [1] Francesco Blefari, Cristian Cosentino, Francesco Aurelio Pironti, Angelo Furfaro, and Fabrizio Marozzo. 2026. CyberRAG: An agentic RAG cyber attack classification and reporting tool. *Future Generation Computer Systems* 176 (2026), 108186. <https://doi.org/10.1016/j.future.2025.108186>
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. (April 2024). <https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>
- [3] Yunfan Gao, Yun Xiong, Meng Wang, and Haofen Wang. 2024. Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks. <https://doi.org/10.48550/arXiv.2407.21059>
- [4] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv:2405.14831 [cs.CL]* <https://arxiv.org/abs/2405.14831>
- [5] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *NeurIPS 2025*. <https://www.microsoft.com/en-us/research/publication/retrievalattention-accelerating-long-context-llm-inference-via-vector-retrieval/>